

Hệ thống quản lý bãi đỗ xe sử dụng công nghệ nhận diện biển số xe (LPR)

Tài liệu thiết kế hệ thống

Thành viên nhóm:

23020417	Nguyễn Minh Quân
23020427	Vũ Văn Tới
23020439	Nguyễn Năng Thịnh
23020433	Mai Phan Anh Tùng
23020395	Nguyễn Văn Linh

Mục lục

1	Giới thiệu chung	3
1.1	Mục đích tài liệu	3
1.2	Phạm vi hệ thống	3
1.3	Đối tượng sử dụng tài liệu	3
1.4	Từ viết tắt và định nghĩa	3
2	Yêu cầu chức năng	4
2.1	Yêu cầu chức năng	4
2.2	Yêu cầu phi chức năng	4
2.3	Các ràng buộc hệ thống	4
3	Thiết kế kiến trúc	5
3.1	Kiến trúc	5
3.2	Các thành phần chính của hệ thống	6
3.2.1	Frontend – Giao diện người dùng	6
3.2.2	Backend – Hệ thống xử lý trung tâm	7
3.2.3	Services – Dịch vụ xử lý hình ảnh thông minh	7
3.3	Cơ sở dữ liệu	7
3.4	Luồng hoạt động tổng quan	8
3.5	Biểu đồ ca sử dụng	8
4	Thiết kế cơ sở dữ liệu	9
4.1	Bảng users – Thông tin người dùng	9
4.2	Bảng parking_config – Cấu hình bãi đỗ xe	10
4.3	Bảng parking_session – Phiên gửi xe	10
5	Thiết kế giao diện	11
5.1	Chi tiết thiết kế	11
5.1.1	Trang chủ (Giới thiệu hệ thống)	11
5.1.2	Trang đăng nhập	12
5.1.3	Trang người dùng (User Dashboard)	12
5.1.4	Trang quản trị (Admin Dashboard)	13
5.2	Ghi chú tổng quan về thiết kế	14
6	Thiết kế tính năng	14
6.1	Tính năng của người quản lý (Admin)	14
6.1.1	Thêm người dùng mới	14
6.1.2	Đăng nhập tài khoản	15
6.1.3	Đăng xuất tài khoản	17
6.1.4	Thay đổi thông tin	18
6.1.5	Xóa người dùng	19
6.1.6	Xem số lượng xe ra vào hàng tháng	20
6.1.7	Vô hiệu hóa tài khoản người dùng	21
6.1.8	Cập nhật phí dịch vụ	22
6.1.9	Xem nhật ký hoạt động của hệ thống	23
6.1.10	Xuất nhật ký hoạt động của hệ thống	24
6.2	Tính năng của người dùng (User)	25

6.2.1	Đăng nhập vào tài khoản User	25
6.2.2	Đăng xuất tài khoản User	26
6.2.3	Nhận diện biển số xe bằng Camera	27
7	Thiết kế API	28
8	Triển khai và vận hành	29
8.1	Môi trường triển khai	29
8.1.1	Môi trường phát triển (Development)	29
8.1.2	Môi trường kiểm thử (Staging/Test)	30
8.1.3	Môi trường sản xuất (Production)	30
8.2	Công cụ đi kèm	30
8.3	Chiến lược backup, giám sát và bảo trì	30
8.3.1	Backup dữ liệu	30
8.3.2	Giám sát hệ thống	30
8.3.3	Bảo trì hệ thống	31
9	Hiệu suất và khả năng mở rộng hệ thống	31
9.1	Hiệu suất hệ thống	31
9.2	Khả năng mở rộng	31

1 Giới thiệu chung

1.1 Mục đích tài liệu

Tài liệu này trình bày thiết kế hệ thống quản lý bãi đỗ xe sử dụng công nghệ nhận diện biển số xe (LPR). Hệ thống hỗ trợ người quản lý kiểm soát ra/vào, thu phí, cấu hình bãi đỗ, thống kê và lưu nhật ký toàn bộ hoạt động một cách minh bạch, hiệu quả.

1.2 Phạm vi hệ thống

Hệ thống bãi đỗ xe bằng nhận diện biển số được phát triển nhằm tự động hóa và tối ưu hóa quá trình quản lý ra/vào xe trong các bãi đỗ. Hệ thống cho phép nhân viên vận hành và người quản trị theo dõi, kiểm soát phương tiện ra vào một cách nhanh chóng, chính xác và minh bạch thông qua công nghệ nhận diện biển số (LPR – License Plate Recognition).

Hệ thống bao gồm hai phân quyền chính:

- **Admin (Người quản trị hệ thống):**
 - Xem tổng quan thống kê người dùng, xe, doanh thu.
 - Quản lý tài khoản người dùng.
 - Quản lý phí dịch vụ.
 - Theo dõi và lọc nhật ký hoạt động toàn hệ thống.
 - Cấu hình các loại bãi đỗ (ô tô, xe máy...).
- **User (Nhân viên vận hành):**
 - Kết nối camera giám sát biển số xe.
 - Nhận diện biển số xe ra/vào.
 - Gửi thông tin lên hệ thống để lưu nhật ký và xử lý thanh toán.

Hệ thống được thiết kế để triển khai trong môi trường nội bộ của các tòa nhà, doanh nghiệp, bãi giữ xe công cộng, với khả năng mở rộng để tích hợp thêm các tính năng nâng cao như cảnh báo phương tiện lạ, phân tích lưu lượng xe, kết nối hệ thống thanh toán tự động hoặc đồng bộ với các phần mềm quản lý khác.

1.3 Đối tượng sử dụng tài liệu

- Admin hệ thống: theo dõi, kiểm tra, cấu hình và giám sát toàn bộ hoạt động bãi đỗ.
- Nhân viên vận hành (User): vận hành camera, nhận diện biển số, hỗ trợ xe ra/vào.
- Nhóm phát triển: xây dựng phần mềm dựa trên yêu cầu thiết kế.
- Người phụ trách IT: triển khai hạ tầng, camera và server vận hành.

1.4 Từ viết tắt và định nghĩa

- **LPR:** License Plate Recognition – công nghệ nhận diện biển số xe.

- **User:** Nhân viên vận hành camera.
- **Admin:** Người quản lý hệ thống tổng thể.
- **Nhật ký hệ thống:** ghi lại toàn bộ hoạt động như tạo tài khoản, nhận diện xe, thanh toán, cấu hình.

2 Yêu cầu chức năng

2.1 Yêu cầu chức năng

Đối với Admin:

- **Trang tổng quan:** Hiển thị các chỉ số quan trọng như tổng số lượt xe ra vào, doanh thu thu được, số lượng người dùng đang hoạt động. Có biểu đồ trực quan thể hiện tần suất hoạt động của hệ thống theo thời gian (ngày, tuần, tháng).
- **Quản lý người dùng:** Tạo, chỉnh sửa, phân quyền và xóa người dùng. Hệ thống đảm bảo mỗi người dùng có quyền phù hợp với vai trò (User hoặc Admin).
- **Quản lý phí dịch vụ:** Thiết lập mức phí theo từng loại phương tiện (ô tô, xe máy,...).
- **Theo dõi nhật ký hệ thống:** Ghi nhận toàn bộ hoạt động trên hệ thống như nhận diện xe, cập nhật tài khoản, điều chỉnh cấu hình, giúp việc giám sát minh bạch.
- **Quản lý báo cáo và thống kê:** Cho phép lọc dữ liệu và xuất báo cáo về lượt xe ra/vào, tổng tiền thu được trong khoảng thời gian cụ thể.

Đối với User:

- **Giao diện camera nhận diện:** Hiển thị luồng hình ảnh trực tiếp từ camera. Cho phép bắt đầu hoặc dừng quá trình nhận diện biển số theo nhu cầu vận hành.
- **Nhận diện biển số:** Khi có xe đi vào hoặc ra khỏi khu vực, hệ thống tự động trích xuất biển số xe từ hình ảnh, lưu lại thời gian nhận diện và loại phương tiện.
- **Tương tác với hệ thống:** Sau khi nhận diện thành công, dữ liệu được gửi lên hệ thống để lưu trữ, đồng thời được dùng cho tính phí và cập nhật nhật ký.
- **Trạng thái nhận diện:** Hệ thống báo rõ trạng thái nhận diện (thành công, thất bại, đang xử lý) để nhân viên theo dõi kịp thời.

2.2 Yêu cầu phi chức năng

- **Thời gian xử lý:** Mỗi lần nhận diện hoàn tất trong ≤ 4 giây.
- **Bảo mật:** Sử dụng JWT, mã hóa mật khẩu.
- **Tính ổn định:** Hoạt động 24/7.

2.3 Các ràng buộc hệ thống

- Camera phải đủ chất lượng để nhận diện rõ biển số.
- Dữ liệu lưu trữ phải bảo mật và phân quyền rõ ràng.
- Nhật ký hệ thống không được xóa trừ khi có quyền cao nhất.

3 Thiết kế kiến trúc

3.1 Kiến trúc

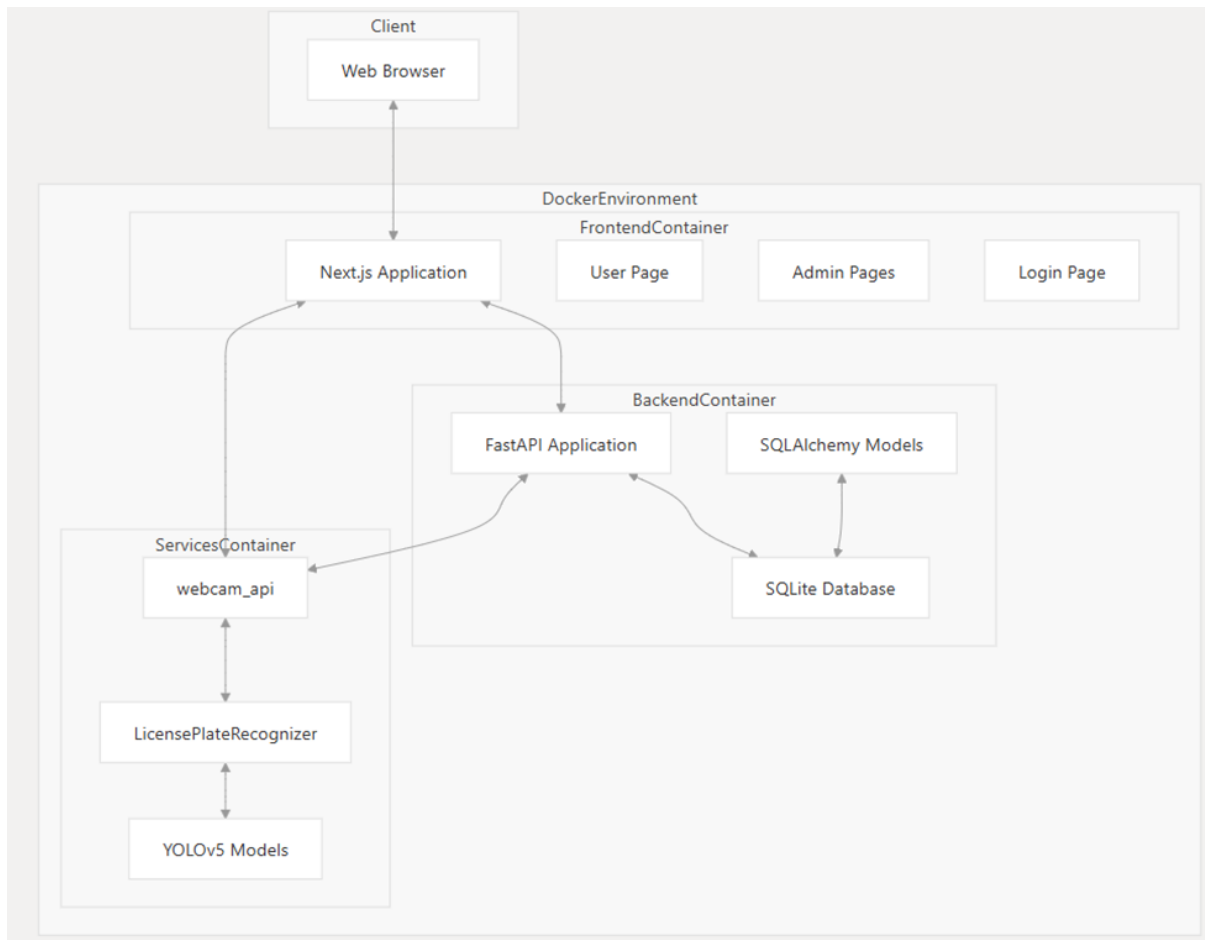
Dự án áp dụng kiến trúc **Client-Server** làm nền tảng, đồng thời phát triển lên **Microservices** nhằm đảm bảo khả năng mở rộng linh hoạt, triển khai độc lập và phát triển đồng bộ giữa các thành phần. Toàn bộ hệ thống được chia thành ba phân lớp chính, mỗi lớp đảm nhiệm một vai trò riêng biệt trong chuỗi xử lý:

- **Frontend:** Giao diện người dùng được xây dựng bằng Next.js, hoạt động trực tiếp trên trình duyệt của người dùng. Đây là tầng giao tiếp, cho phép người dùng thực hiện các thao tác như gửi xe, truy xuất danh sách xe, danh sách người dùng. Thông qua việc gửi các truy vấn (request) tới backend, frontend đóng vai trò cầu nối giữa người dùng và hệ thống xử lý dữ liệu phía sau.
- **Backend:** Lõi xử lý chính của hệ thống được phát triển bằng FastAPI – một framework hiện đại, gọn nhẹ, cho phép xây dựng các API RESTful với hiệu suất cao. Backend sử dụng SQLAlchemy để ánh xạ giữa cơ sở dữ liệu quan hệ và các thực thể Python, cùng với SQLite làm hệ quản trị cơ sở dữ liệu nội bộ. Kiến trúc backend được thiết kế mở để dễ dàng nâng cấp lên các hệ quản trị cơ sở dữ liệu mạnh hơn khi cần thiết, mà không ảnh hưởng đến kiến trúc tổng thể.
- **Services:** Thành phần chuyên trách cho các tác vụ chuyên sâu, cụ thể là xử lý hình ảnh. Module này sử dụng mô hình YOLOv5 để nhận diện hình ảnh xe cộ và biển số, triển khai thông qua FastAPI để duy trì tính đồng nhất với backend chính. Services hoạt động độc lập, nhận yêu cầu từ backend, thực hiện xử lý và trả kết quả thông qua các API trung gian.

Để tăng hiệu quả hoạt động, hệ thống tích hợp cơ chế xử lý bất đồng bộ với từ khóa `async/await`. Điều này cho phép backend thực hiện các tác vụ I/O như truy vấn cơ sở dữ liệu, gọi API giữa các service một cách song song và không chặn luồng chính, giúp cải thiện đáng kể hiệu suất phản hồi và khả năng mở rộng khi có nhiều người dùng truy cập đồng thời.

Với mô hình kiến trúc hiện đại và phân tách rõ ràng, dự án đảm bảo tính hiệu quả trong vận hành, đồng thời hỗ trợ khả năng mở rộng quy mô, bảo trì dễ dàng và phát triển lâu dài trong môi trường thực tế.

Dưới đây là mô hình của hệ thống :



3.2 Các thành phần chính của hệ thống

Hệ thống được xây dựng theo kiến trúc **Client-Server**, mở rộng theo mô hình **Microservices** để tách biệt chức năng, tăng khả năng mở rộng và triển khai linh hoạt. Các thành phần chính gồm:

3.2.1 Frontend – Giao diện người dùng

- **Công nghệ sử dụng:** [Next.js](#).
- Đóng vai trò là tầng giao tiếp giữa người dùng và hệ thống.
- Chịu trách nhiệm gửi yêu cầu HTTP đến Backend và hiển thị thông tin trả về.

Gồm 2 giao diện chính:

1. Giao diện Người dùng (User Interface):

- Đăng nhập
- Quét biển số xe (camera hoặc upload ảnh)
- Xem thông tin xe gửi
- Đăng xuất

2. Giao diện Quản trị (Admin Interface):

- Đăng nhập quản trị
- Trang tổng quan (dashboard)
- Quản lý người dùng
- Cấu hình/Chỉnh sửa phí gửi xe theo loại phương tiện
- Xem lịch sử gửi xe, nhật ký (logs), thống kê

3.2.2 Backend – Hệ thống xử lý trung tâm

- **Công nghệ sử dụng:** [FastAPI](#).
- Đóng vai trò trung gian giữa frontend, cơ sở dữ liệu và các services nhận diện.

Chức năng chính:

- Xác thực và phân quyền người dùng
- Mã hóa và kiểm tra mật khẩu bằng thuật toán an toàn
- Xử lý truy vấn đến cơ sở dữ liệu để lấy thông tin xe, người dùng, lịch sử gửi xe
- Tổng hợp dữ liệu để phục vụ cho các yêu cầu hiển thị hoặc thống kê của frontend
- Gọi tới các services phụ trợ (ví dụ: dịch vụ nhận diện biển số)

3.2.3 Services – Dịch vụ xử lý hình ảnh thông minh

- **Công nghệ sử dụng:**
 - [FastAPI](#): triển khai service
 - [YOLOv5](#): mô hình học sâu nhận diện đối tượng
 - WebSocket: truyền dữ liệu thời gian thực

Chức năng chính:

- Nhận hình ảnh từ camera hoặc từ yêu cầu frontend
- Xử lý ảnh, phát hiện biển số xe, trích xuất thông tin bằng YOLOv5
- Gửi kết quả nhận diện về backend qua API
- Truyền kết quả nhận diện thời gian thực qua WebSocket để frontend cập nhật trực tiếp
- Hỗ trợ chức năng check-in/check-out cho xe gửi dựa trên biển số

3.3 Cơ sở dữ liệu

- **Hệ quản trị CSDL:** [SQLite](#).
- **ORM sử dụng:** [SQLAlchemy](#).
- Thiết kế theo mô hình quan hệ (Relational Model).

Thông tin được lưu trữ bao gồm:

- Dữ liệu người dùng: thông tin xác thực, vai trò, trạng thái
- Thông tin xe: loại xe, biển số, người sở hữu

- Phiên gửi xe: thời gian vào/ra, phí, trạng thái
- Cấu hình hệ thống: giá dịch vụ theo loại xe
- Lịch sử tương tác: nhật ký, logs hệ thống

3.4 Luồng hoạt động tổng quan

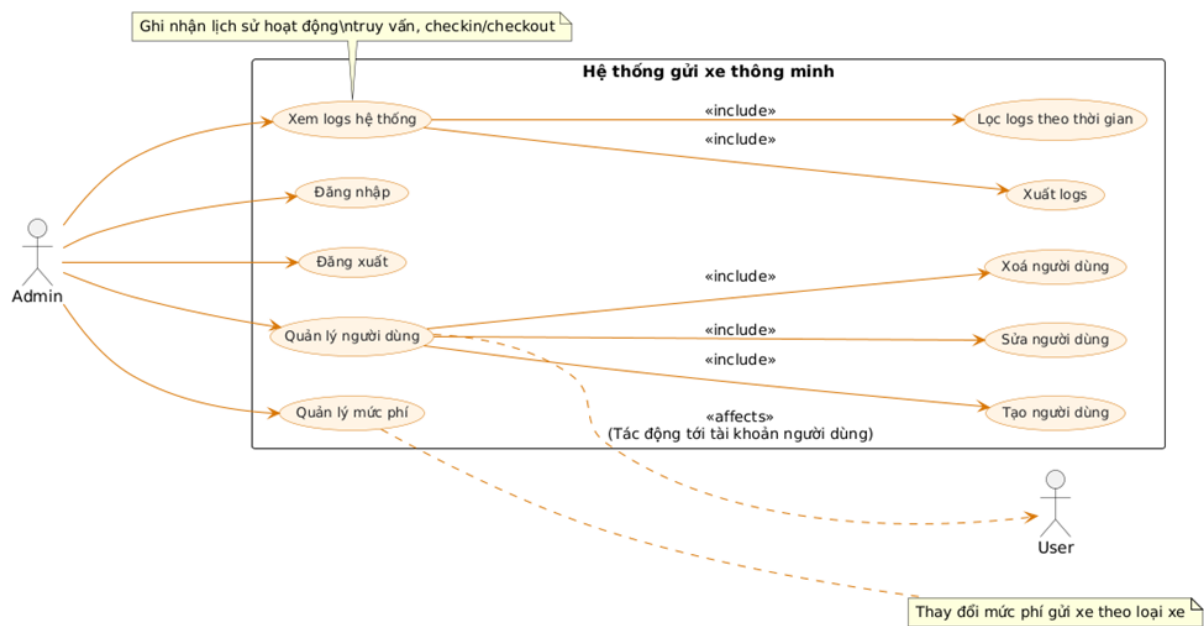
Hệ thống hoạt động theo mô hình tương tác đa lớp, phối hợp giữa client, backend và service nhận diện.

Luồng xử lý chính:

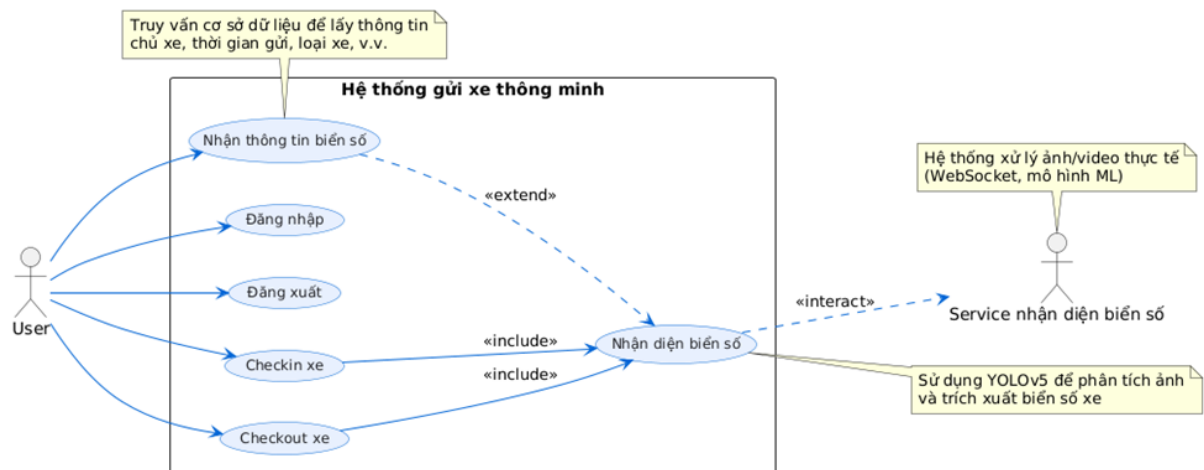
1. Người dùng (hoặc quản trị viên) tương tác với **Frontend** (Next.js) thông qua trình duyệt.
2. **Frontend** gửi yêu cầu HTTP tới **Backend** và/hoặc **Services**, kèm theo dữ liệu (thông tin xe,...).
3. **Backend** và **Services** tiếp nhận, xử lý:
 - **Backend:**
 - Xác thực người dùng và kiểm tra quyền
 - Truy vấn hoặc cập nhật cơ sở dữ liệu (biển số, người dùng, phiên gửi xe...)
 - **Services:**
 - Nhận ảnh, xử lý nhận diện biển số bằng YOLOv5
 - Truyền kết quả về qua API hoặc WebSocket
 - Gọi **Backend** xử lý check-in/check-out theo biển số xe
4. **Backend** và **Services** trả kết quả (dữ liệu, thông tin nhận diện, trạng thái) về cho **Frontend**.
5. **Frontend** hiển thị kết quả theo thời gian thực cho người dùng.

3.5 Biểu đồ ca sử dụng

Ca sử dụng cho admin :



Ca sử dụng cho user :



4 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng cơ sở dữ liệu SQLite, kết hợp với SQLAlchemy giúp ánh xạ dữ liệu từ SQL sang Python giúp quản lý, sử dụng dễ dàng hơn. Cấu trúc cơ sở dữ liệu được thiết kế theo mô hình thực thể - quan hệ.

4.1 Bảng users – Thông tin người dùng

Trường	Kiểu dữ liệu	Mô tả	Ràng buộc
id	Integer	Định danh duy nhất của người dùng	Primary Key, Tự động tăng
username	String	Tên đăng nhập	Unique, Indexed, Không trùng lặp

email	String	Địa chỉ email	Unique, Indexed, Không trùng lặp
hashed_password	String	Mật khẩu đã được mã hóa an toàn	Not Null
role	Enum	Vai trò người dùng trong hệ thống	Mặc định: USER
is_active	Boolean	Trạng thái hoạt động của tài khoản	Mặc định: True
created_at	DateTime	Thời điểm tạo tài khoản	Mặc định: Thời gian hiện tại (giờ VN)

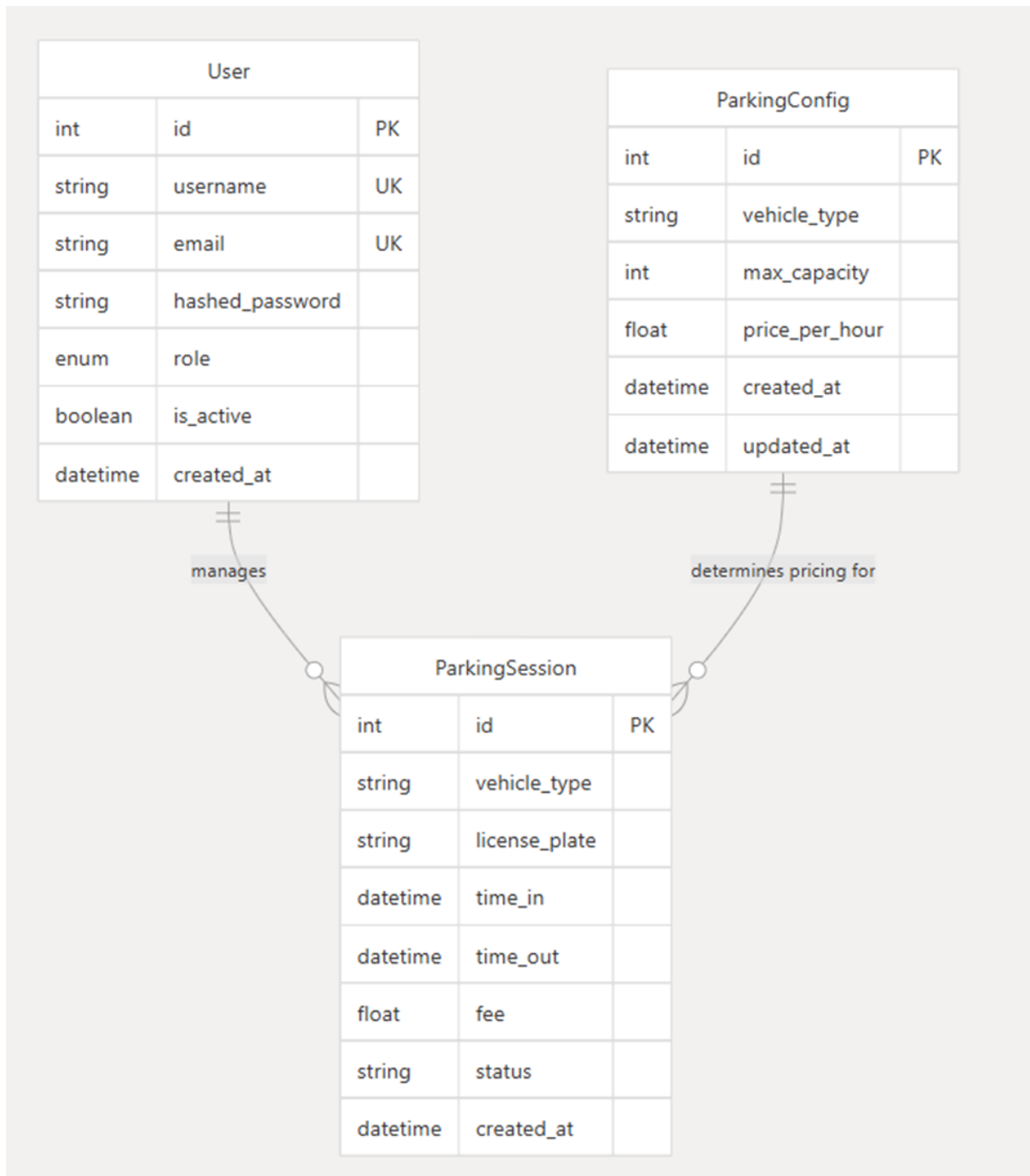
4.2 Bảng parking_config – Cấu hình bãi đỗ xe

Trường	Kiểu dữ liệu	Mô tả	Ràng buộc
id	Integer	Định danh duy nhất	Primary Key, Tự động tăng
vehicle_type	String	Loại phương tiện	Not Null
max_capacity	Integer	Sức chứa tối đa	Not Null
price_per_hour	Float	Đơn giá theo giờ	Not Null
created_at	DateTime	Thời điểm tạo cấu hình	Mặc định: Thời gian hiện tại (giờ VN)
updated_at	DateTime	Thời điểm cập nhật cấu hình	Tự động cập nhật

4.3 Bảng parking_session – Phiên gửi xe

Trường	Kiểu dữ liệu	Mô tả	Ràng buộc
id	Integer	Định danh duy nhất của phiên gửi xe	Primary Key, Tự động tăng
vehicle_type	String	Loại phương tiện được gửi	Not Null
license_plate	String	Biển số xe	Not Null
time_in	DateTime	Thời điểm vào bãi	Mặc định: Thời gian hiện tại (giờ VN)
time_out	DateTime	Thời điểm ra khỏi bãi	Cho phép NULL
fee	Float	Phí gửi xe được tính toán	Cho phép NULL
status	String	Trạng thái phiên gửi xe	Mặc định: "active"
created_at	DateTime	Thời điểm ghi nhận dữ liệu	Mặc định: Thời gian hiện tại (giờ VN)

Sơ đồ thực thể liên kết :



5 Thiết kế giao diện

5.1 Chi tiết thiết kế

5.1.1 Trang chủ (Giới thiệu hệ thống)

- **Name:** Trang giới thiệu tổng quan
- **Description:** Cung cấp thông tin khái quát về hệ thống nhận diện biển số xe, mục đích sử dụng và các tính năng chính.

- **Route:** /
- **File:** app/page.tsx
- **Components:**
 - **HeroSection:** Tiêu đề lớn giới thiệu hệ thống.
 - **FeatureOverview:** Các tính năng nổi bật (nhận diện biển số tự động, bảo mật cao, quản lý dễ dàng,...).
 - **CallToAction:** Nút điều hướng đến trang đăng nhập.
- **Layout:**
 - Tiêu đề: "Giải pháp quản lý bãi đỗ xe thông minh"
 - Mô tả: "Hệ thống nhận diện biển số xe tự động, giúp quản lý bãi đỗ xe hiệu quả và tiết kiệm thời gian"
 - Thiết kế: Nền trắng, bố cục thoáng, icon minh họa đơn giản.

5.1.2 Trang đăng nhập

- **Name:** Trang đăng nhập vào hệ thống
- **Description:** Giao diện cho phép người dùng truy cập hệ thống bằng thông tin đăng nhập.
- **Route:** /login
- **File:** app/login/page.tsx
- **Components:**
 - **LoginForm:**
 - * Trường nhập tên đăng nhập (username)
 - * Trường nhập mật khẩu (password)
 - * Nút "Đăng nhập"
 - * Thông báo lỗi nếu thông tin không hợp lệ
- **Layout:**
 - Tiêu đề: "Đăng nhập"
 - Mô tả: "Nhập thông tin đăng nhập để truy cập vào hệ thống"
 - Thiết kế: Form đăng nhập căn giữa màn hình, nền trắng, bo góc, bóng đổ nhẹ. Nút "Đăng nhập" màu xanh nước biển, nổi bật. Giao diện responsive.

5.1.3 Trang người dùng (User Dashboard)

- **Name:** Dashboard dành cho người dùng
- **Description:** Giao diện chính dành cho người dùng, bao gồm camera nhận diện biển số và phần hiển thị thông tin biển số đã nhận diện.
- **Route:** /user
- **File:** app/user/page.tsx

- **Components:**
 - **CameraRecognition:** Giao diện camera nhận diện biển số, hiển thị trực tiếp hình ảnh từ camera.
 - **PlateInfoDisplay:** Phần hiển thị chi tiết thông tin biển số đã nhận diện.
 - **UserToolBar:** Chứa nút đăng xuất nằm trên cùng bên phải màn hình.
- **Layout:**
 - Thanh công cụ trên cùng góc phải chứa **UserToolBar** để thao tác.
 - Giao diện chia thành hai phần chính:
 - * Bên trái: **CameraRecognition** – hiển thị camera nhận diện biển số.
 - * Bên phải: **PlateInfoDisplay** – hiển thị thông tin chi tiết biển số.
 - Thiết kế responsive: Trên thiết bị nhỏ, hai phần sẽ xếp dọc để tối ưu hiển thị.

5.1.4 Trang quản trị (Admin Dashboard)

- **Name:** Dashboard dành cho quản trị viên
- **Description:** Giao diện quản lý toàn bộ hệ thống qua các chức năng chính được tổ chức thành các tab rõ ràng, giúp admin dễ dàng quản lý người dùng, phí dịch vụ, nhật ký và tài khoản.
- **Route:** /admin
- **File:** app/admin/page.tsx
- **Components chính:**
 - **AdminToolbar:**
 - * Nút “Đăng xuất”
 - **SidebarNavigation** với các mục:
 - * **OverviewTab:** Hiển thị thông tin tổng quan và thống kê hệ thống (tổng người dùng, xe đã quản lý, doanh thu,...).
 - * **UserManagementTab:**
 - Danh sách người dùng
 - Chức năng tạo, sửa, xóa người dùng
 - Phân quyền người dùng
 - * **ServiceFeeTab:** Quản lý và cấu hình các loại phí dịch vụ.
 - * **SystemLogTab:** Xem và lọc nhật ký hoạt động hệ thống.
 - * **AdminAccountTab:** Quản lý tài khoản admin, thay đổi mật khẩu, thông tin cá nhân.
- **Layout:**
 - Thanh công cụ **AdminToolbar** đặt ở vị trí trên cùng bên phải, cố định, chứa nút “Đăng xuất”.

- **SidebarNavigation** nằm phía ngoài cùng bên trái, cho phép chuyển đổi linh hoạt giữa các chức năng.
- Phần nội dung chính thay đổi tương ứng theo mục đang chọn, bố cục rõ ràng, dễ thao tác.
- Giao diện responsive, tương thích mọi kích thước màn hình từ desktop đến thiết bị di động.

5.2 Ghi chú tổng quan về thiết kế

- **Design Philosophy:**

- **Phong cách:** Hiện đại, tối giản, sử dụng tông màu chủ đạo trắng, xanh dương.
- **Tương thích:** Responsive trên desktop và mobile.
- **Trải nghiệm người dùng:**
 - * Dễ hiểu, thao tác nhanh chóng
 - * Thông báo rõ ràng khi có lỗi hoặc thao tác quan trọng
 - * Giao diện admin trực quan, quản lý tập trung
- **Accessibility:** Tuân thủ các tiêu chuẩn thiết kế có thể truy cập (màu sắc, font dễ đọc, hỗ trợ điều hướng bàn phím...).

6 Thiết kế tính năng

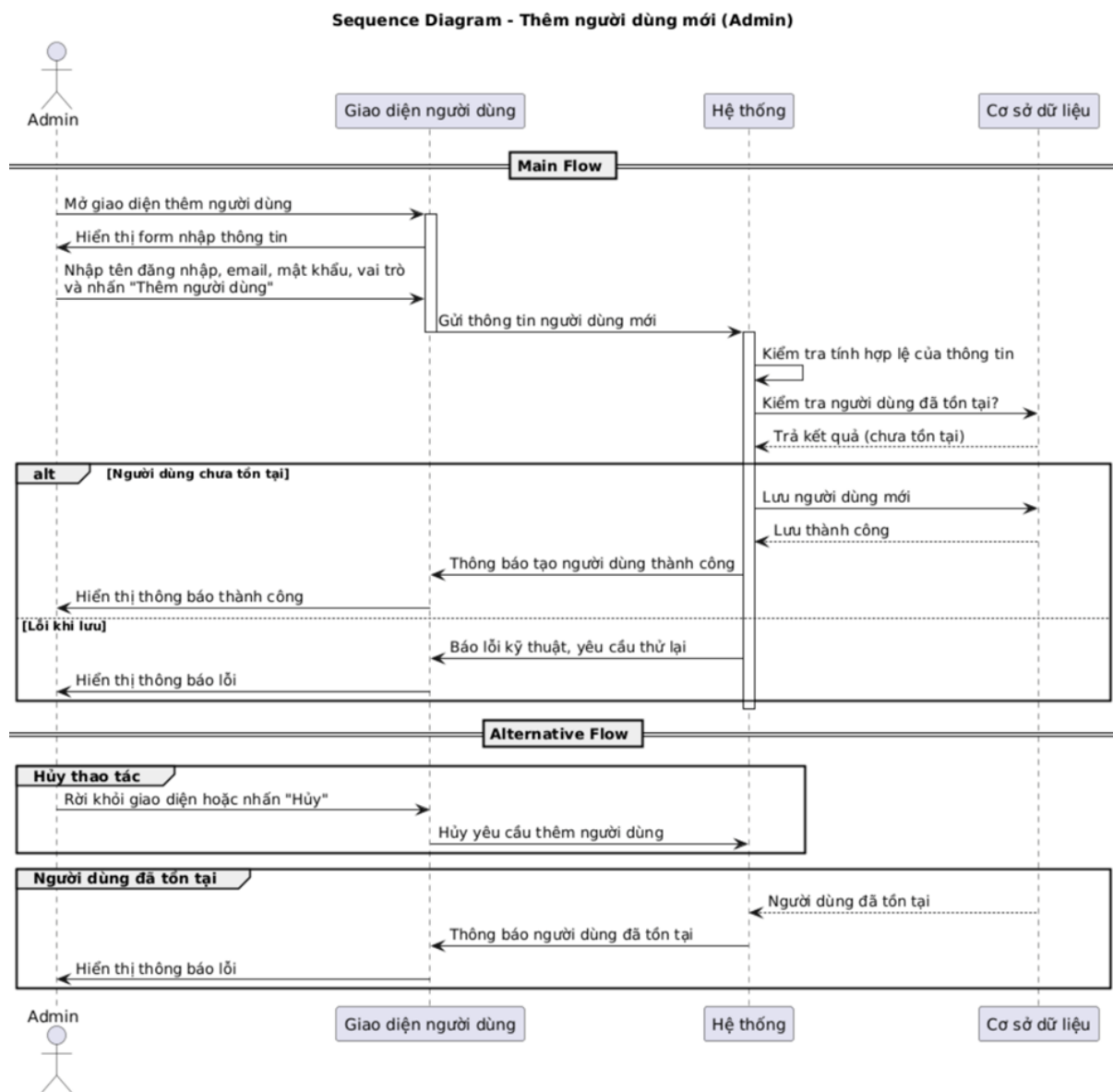
6.1 Tính năng của người quản lý (Admin)

6.1.1 Thêm người dùng mới

- **Name:** Thêm người dùng mới
- **Actor:** Admin
- **Goal:** Admin thêm người dùng mới thành công để sử dụng với vai trò Admin hoặc User.
- **Pre-condition:**
 - Admin đã đăng nhập thành công vào hệ thống.
 - Hệ thống hiển thị giao diện thêm người dùng mới.
- **Main Flow:**
 1. Admin nhập thông tin cần thiết (**Tên đăng nhập, email, mật khẩu, vai trò của tài khoản**) và nhấn nút thêm người dùng.
 2. Hệ thống kiểm tra tính hợp lệ của thông tin.
 3. Hệ thống kiểm tra xem người dùng đã tồn tại trong cơ sở dữ liệu hay chưa.
 4. Hệ thống lưu thông tin người dùng mới vào cơ sở dữ liệu.
 5. Hệ thống tạo người dùng mới thành công.

- **Alternative Flow:**

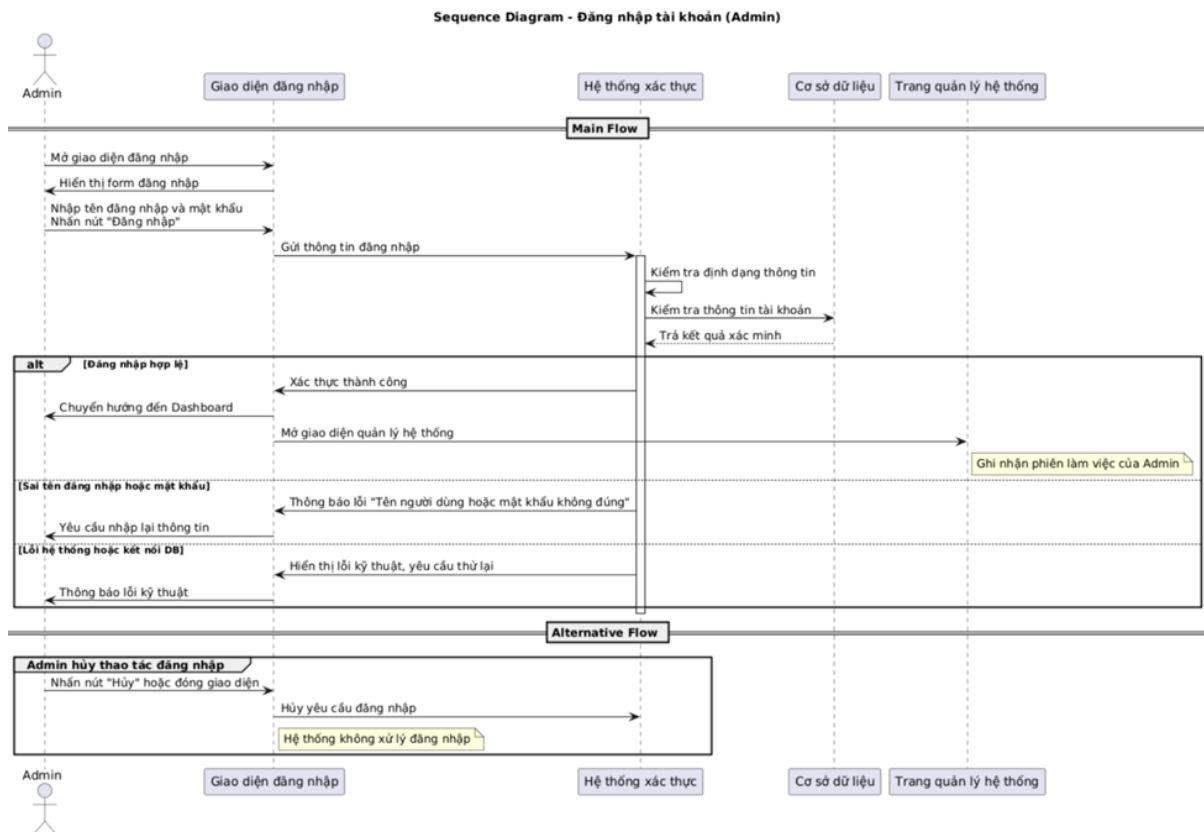
- Bước 1: Nếu Admin hủy thêm người dùng mới hoặc chuyển hướng sang trang khác trước khi nhấn nút thêm người dùng, hệ thống hủy bỏ yêu cầu.
- Bước 3: Nếu thông tin người dùng đã tồn tại trong cơ sở dữ liệu, hệ thống không cho tạo người dùng mới.
- Bước 4: Nếu lỗi khi lưu thông tin, hệ thống báo lỗi kỹ thuật và yêu cầu thử lại sau.



6.1.2 Đăng nhập tài khoản

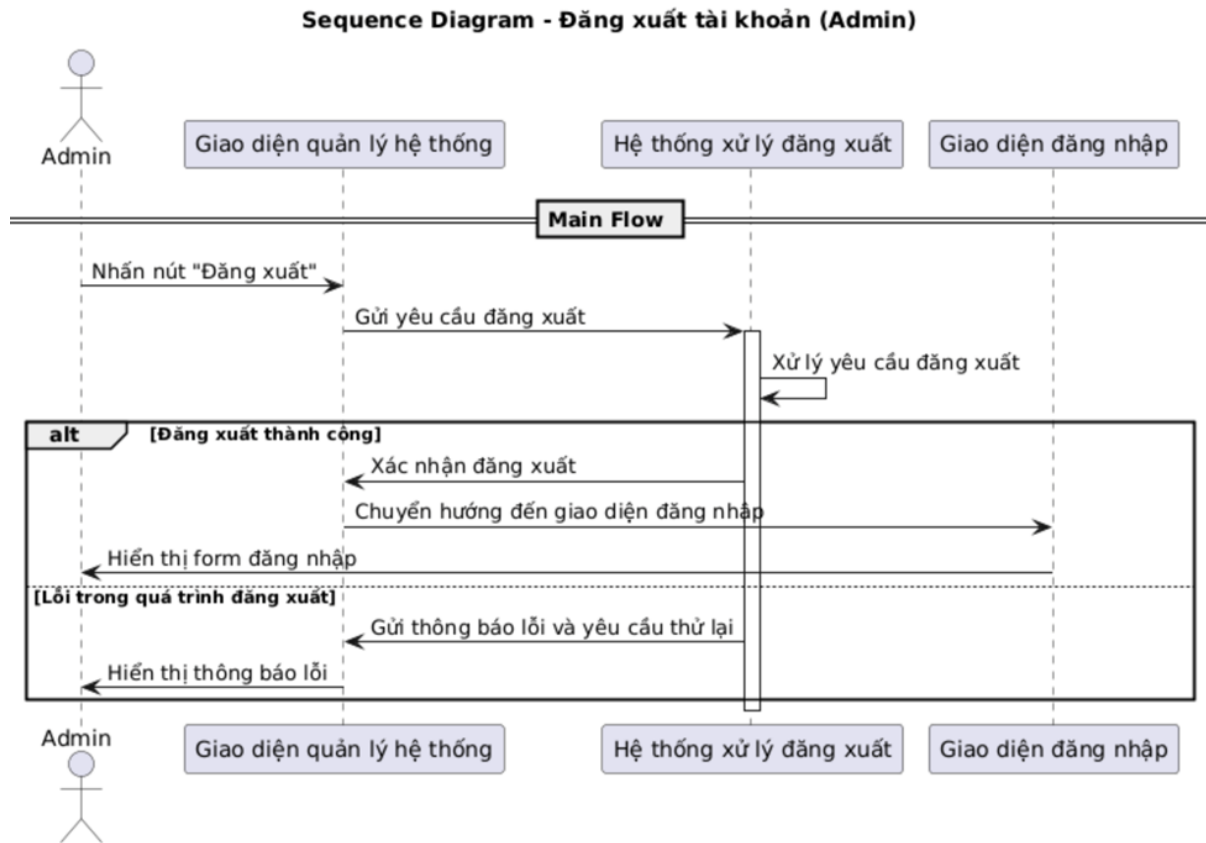
- **Name:** Đăng nhập tài khoản
- **Actor:** Admin
- **Goal:** Admin truy cập được vào hệ thống bằng tài khoản đã được tạo từ trước.
- **Pre-condition:**

- Tài khoản đã được đăng ký từ trước.
- Hệ thống hiển thị giao diện đăng nhập.
- **Main Flow:**
 1. Admin nhập thông tin đăng nhập gồm **tên đăng nhập**, **mật khẩu** và nhấn nút xác nhận đăng nhập.
 2. Hệ thống kiểm tra tính hợp lệ của thông tin.
 3. Hệ thống kiểm tra xem tài khoản có khớp với dữ liệu trong cơ sở dữ liệu hay không.
 4. Hệ thống chuyển hướng đến trang quản lý hệ thống của Admin.
- **Alternative Flow:**
 - Bước 1: Nếu Admin hủy bỏ thao tác đăng nhập, hệ thống hủy bỏ yêu cầu đăng nhập.
 - Bước 3a: Nếu thông tin đăng nhập không tồn tại trong cơ sở dữ liệu hoặc **mật khẩu** hay **tên đăng nhập** chưa đúng, hệ thống báo lỗi “Tên người dùng hoặc mật khẩu không đúng” và yêu cầu nhập lại.
 - Bước 3b: Nếu có lỗi hệ thống hoặc lỗi kết nối đến cơ sở dữ liệu, hệ thống hiển thị thông báo lỗi kỹ thuật và yêu cầu thử lại sau.
- **Post-condition:**
 - Admin đăng nhập thành công và được chuyển đến giao diện quản lý hệ thống.
 - Hệ thống ghi nhận phiên làm việc của Admin.



6.1.3 Đăng xuất tài khoản

- **Name:** Đăng xuất tài khoản
- **Actor:** Admin
- **Goal:** Admin đăng xuất ra khỏi hệ thống.
- **Pre-condition:**
 - Tài khoản của Admin đã đăng nhập vào hệ thống.
 - Hệ thống hiển thị nút có chức năng đăng xuất.
- **Main Flow:**
 1. Admin nhấn vào nút “Đăng xuất”.
 2. Hệ thống xử lý yêu cầu và đăng xuất tài khoản Admin.
 3. Hệ thống chuyển hướng đến giao diện đăng nhập.
- **Alternative Flow:**
 - Bước 2: Nếu xảy ra lỗi trong quá trình đăng xuất, hệ thống hiển thị thông báo lỗi và yêu cầu thử lại.
- **Post-condition:**
 - Admin đã đăng xuất khỏi hệ thống.
 - Hệ thống hiển thị giao diện đăng nhập.



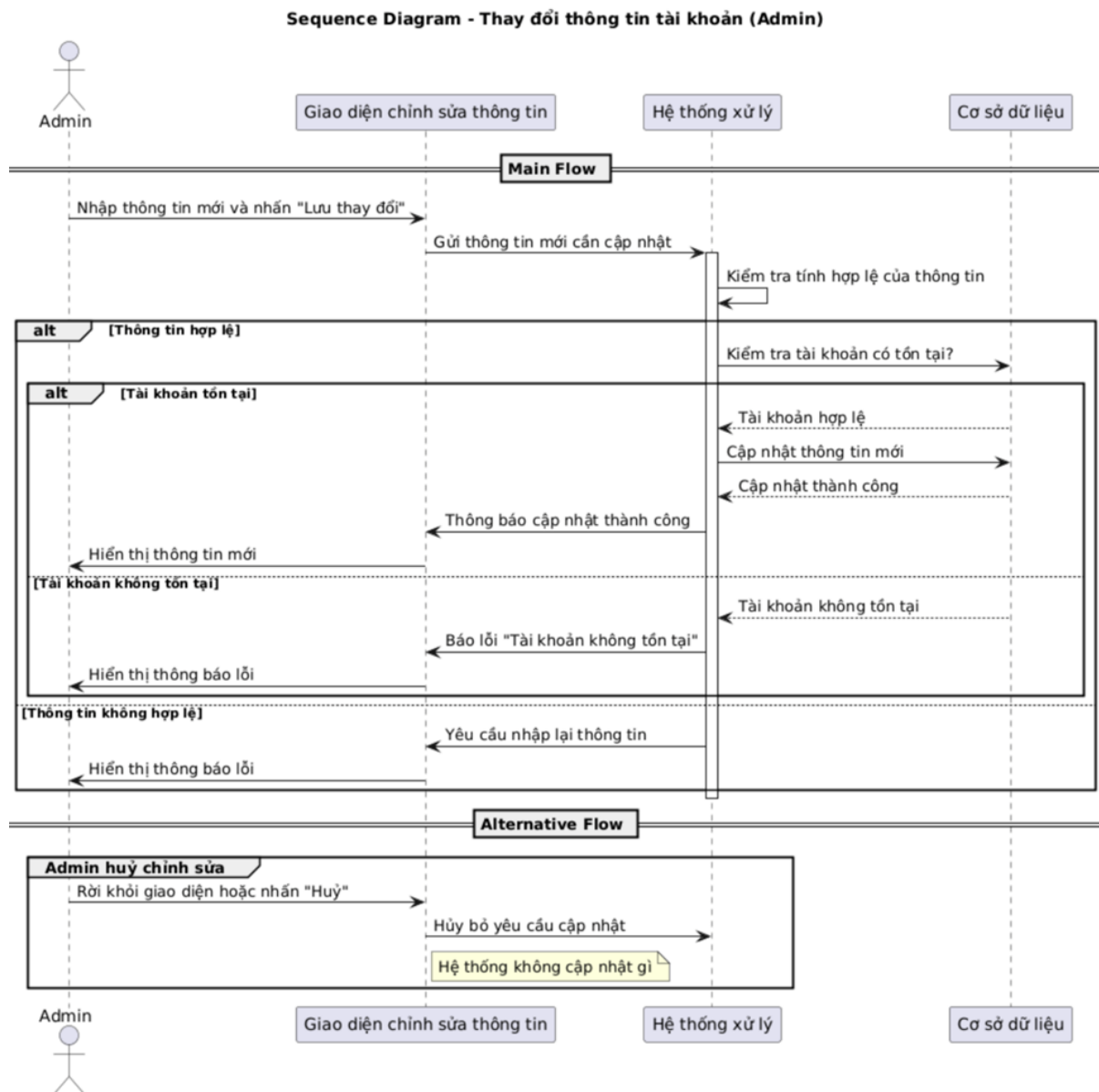
6.1.4 Thay đổi thông tin

- **Name:** Thay đổi thông tin của tài khoản
- **Actor:** Admin
- **Goal:** Admin chỉnh sửa và cập nhật thành công thông tin tài khoản.
- **Pre-condition:**
 - Admin đã đăng nhập vào hệ thống.
 - Tài khoản hợp lệ và tồn tại trong cơ sở dữ liệu.
 - Hệ thống hiển thị giao diện thay đổi thông tin.
- **Main Flow:**
 1. Admin cập nhật thông tin mới và nhấn nút lưu thay đổi.
 2. Hệ thống kiểm tra tính hợp lệ của thông tin đã nhập.
 3. Hệ thống xác minh rằng tài khoản cần chỉnh sửa có tồn tại trong cơ sở dữ liệu.
 4. Hệ thống lưu thông tin mới vào cơ sở dữ liệu.
 5. Hệ thống thành công thay đổi thông tin tài khoản.
- **Alternative Flow:**
 - Bước 1: Nếu Admin hủy thao tác chỉnh sửa hoặc chuyển sang trang khác trước khi lưu thay đổi, hệ thống hủy bỏ yêu cầu.

- Bước 2: Nếu thông tin không hợp lệ, hệ thống yêu cầu nhập lại.
- Bước 3: Nếu tài khoản không tồn tại trong cơ sở dữ liệu, hệ thống báo lỗi.

• **Post-condition:**

- Thông tin mới của tài khoản được cập nhật thành công vào cơ sở dữ liệu.
- Hệ thống hiển thị thông tin mới trên giao diện.



6.1.5 Xóa người dùng

- **Name:** Xóa người dùng
- **Actor:** Admin
- **Goal:** Người dùng được Admin chọn bị xóa khỏi hệ thống.
- **Pre-condition:**
 - Admin đã đăng nhập vào hệ thống.

- Tài khoản người dùng hợp lệ và đang tồn tại trong cơ sở dữ liệu.
- Hệ thống hiển thị danh sách người dùng và chức năng xóa.

• **Main Flow:**

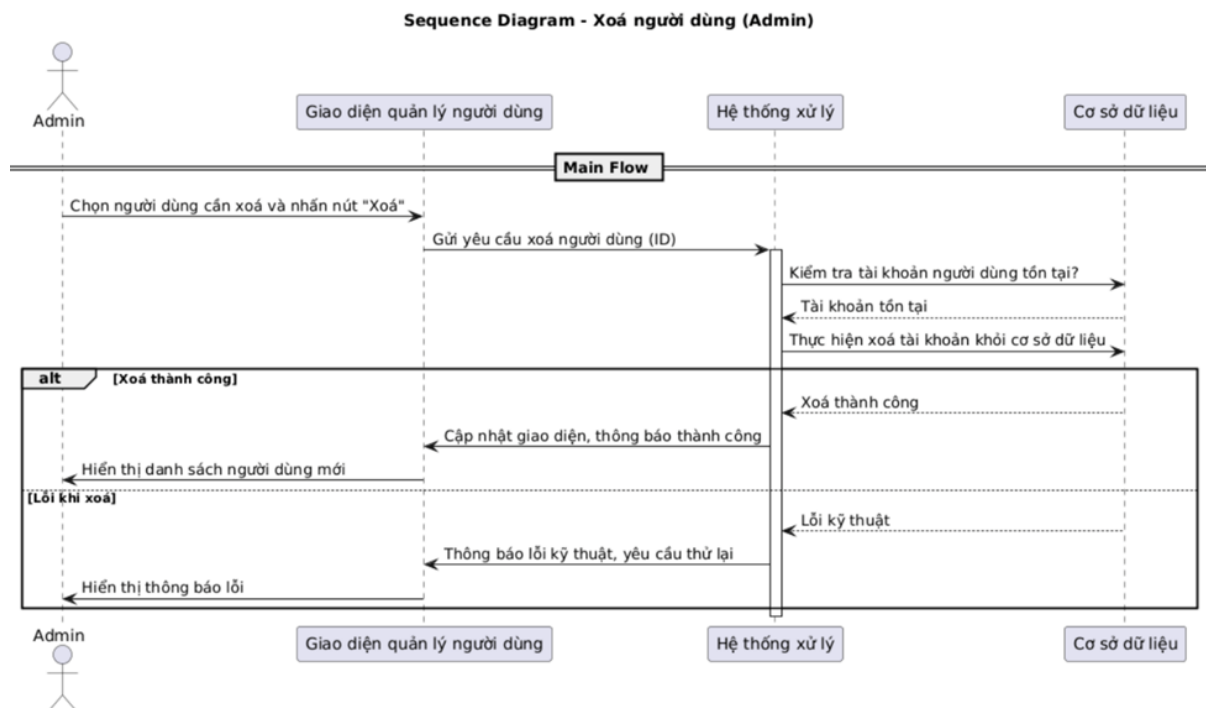
1. Admin chọn người dùng cần xóa và nhấn vào nút xóa người dùng.
2. Hệ thống xác minh tài khoản và thực hiện xóa toàn bộ thông tin người dùng khỏi cơ sở dữ liệu.
3. Hệ thống xóa thành công và trở lại màn hình Admin.

• **Alternative Flow:**

- Bước 2: Nếu xảy ra lỗi khi xóa thông tin trong cơ sở dữ liệu, hệ thống hiển thị lỗi kỹ thuật và yêu cầu thử lại sau.

• **Post-condition:**

- Người dùng bị xóa khỏi hệ thống và không còn tồn tại trong cơ sở dữ liệu.
- Giao diện hệ thống cập nhật lại danh sách người dùng.



6.1.6 Xem số lượng xe ra vào hàng tháng

- **Name:** Xem số lượng xe ra vào hàng tháng
- **Actor:** Admin
- **Goal:** Hệ thống trả về số lượng xe ra vào trong 12 tháng gần nhất cho Admin.
- **Pre-condition:**
 - Admin đã đăng nhập vào hệ thống.
 - Hệ thống hiển thị giao diện màn hình chính của Admin.

- **Main Flow:**

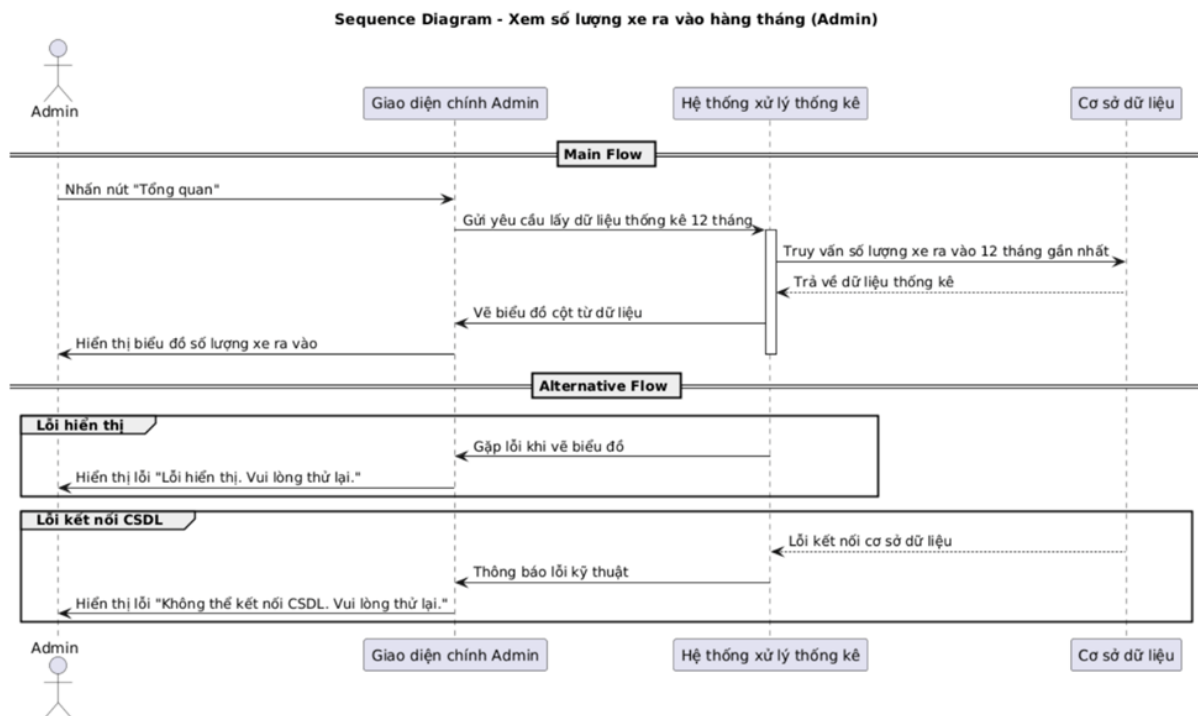
1. Admin nhấn vào nút “Tổng quan”.
2. Hệ thống truy vấn cơ sở dữ liệu và hiển thị biểu đồ cột thể hiện số lượng xe ra vào trong 12 tháng gần nhất.

- **Alternative Flow:**

- Bước 1: Nếu có lỗi trong quá trình hiển thị, hệ thống báo lỗi và yêu cầu thử lại.
- Bước 2: Nếu hệ thống gặp lỗi kết nối cơ sở dữ liệu, hệ thống báo lỗi và yêu cầu thử lại sau.

- **Post-condition:**

- Hệ thống hiển thị biểu đồ cột thể hiện số lượng xe ra vào trong 12 tháng gần nhất.



6.1.7 Vô hiệu hóa tài khoản người dùng

- **Name:** Vô hiệu hóa người dùng

- **Actor:** Admin

- **Goal:** Người dùng bị vô hiệu hóa tài khoản bởi Admin.

- **Pre-condition:**

- Admin đã đăng nhập vào hệ thống.
- Hệ thống hiển thị danh sách các người dùng.

- **Main Flow:**

1. Admin chọn người dùng muốn vô hiệu hóa tài khoản.

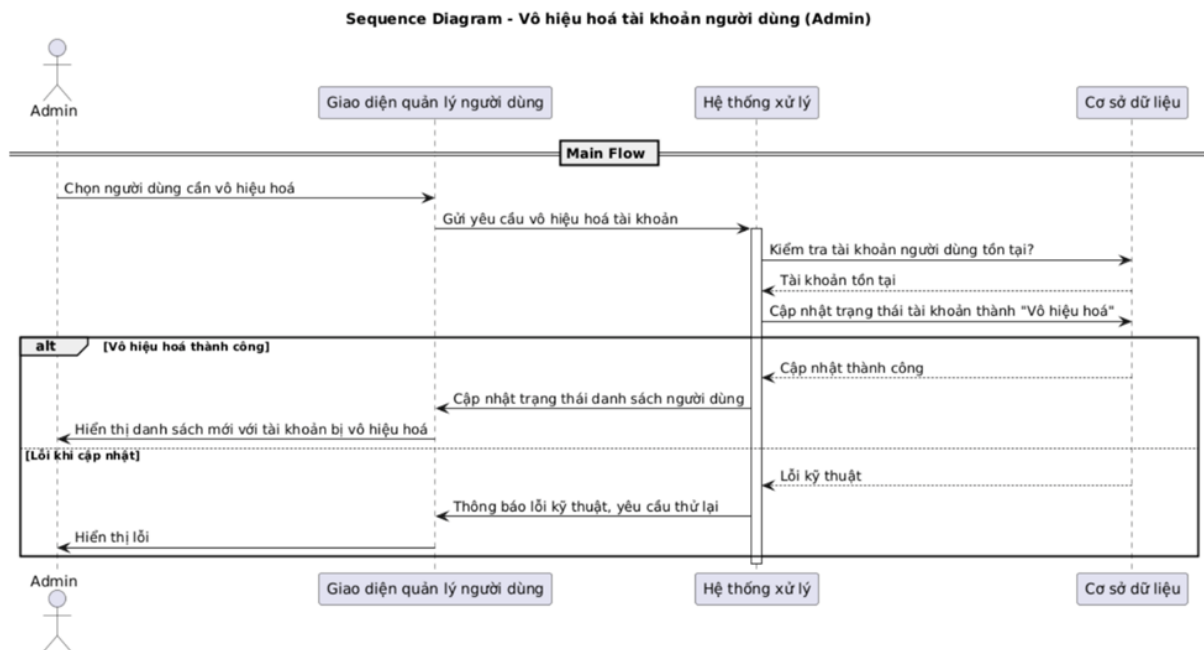
2. Hệ thống xác minh yêu cầu và thực hiện vô hiệu hóa tài khoản được chọn.
3. Hệ thống cập nhật lại danh sách trạng thái của người dùng.
4. Tài khoản được chọn bị vô hiệu hóa và không thể sử dụng được nữa.

- **Alternative Flow:**

- Bước 2: Nếu xảy ra lỗi khi vô hiệu hóa tài khoản trong cơ sở dữ liệu, hệ thống hiển thị lỗi kỹ thuật và yêu cầu thử lại sau.

- **Post-condition:**

- Tài khoản người dùng bị vô hiệu hóa và không thể sử dụng được.
- Giao diện hiển thị danh sách trạng thái mới của người dùng.



6.1.8 Cập nhật phí dịch vụ

- **Name:** Cập nhật phí dịch vụ của bãi đỗ xe
- **Actor:** Admin
- **Goal:** Giá dịch vụ được cập nhật bởi Admin.
- **Pre-condition:**
 - Admin đã đăng nhập vào hệ thống.
 - Hệ thống hiển thị màn hình chính của Admin.
- **Main Flow:**
 1. Admin nhấn vào nút “Phí dịch vụ”.
 2. Admin chọn loại phương tiện muốn thay đổi phí và nhấn nút “Chỉnh sửa” ở dòng tương ứng.
 3. Hệ thống hiển thị ô chỉnh sửa phí dịch vụ, cho phép người dùng thay đổi phí theo giờ, sức chứa tối đa.

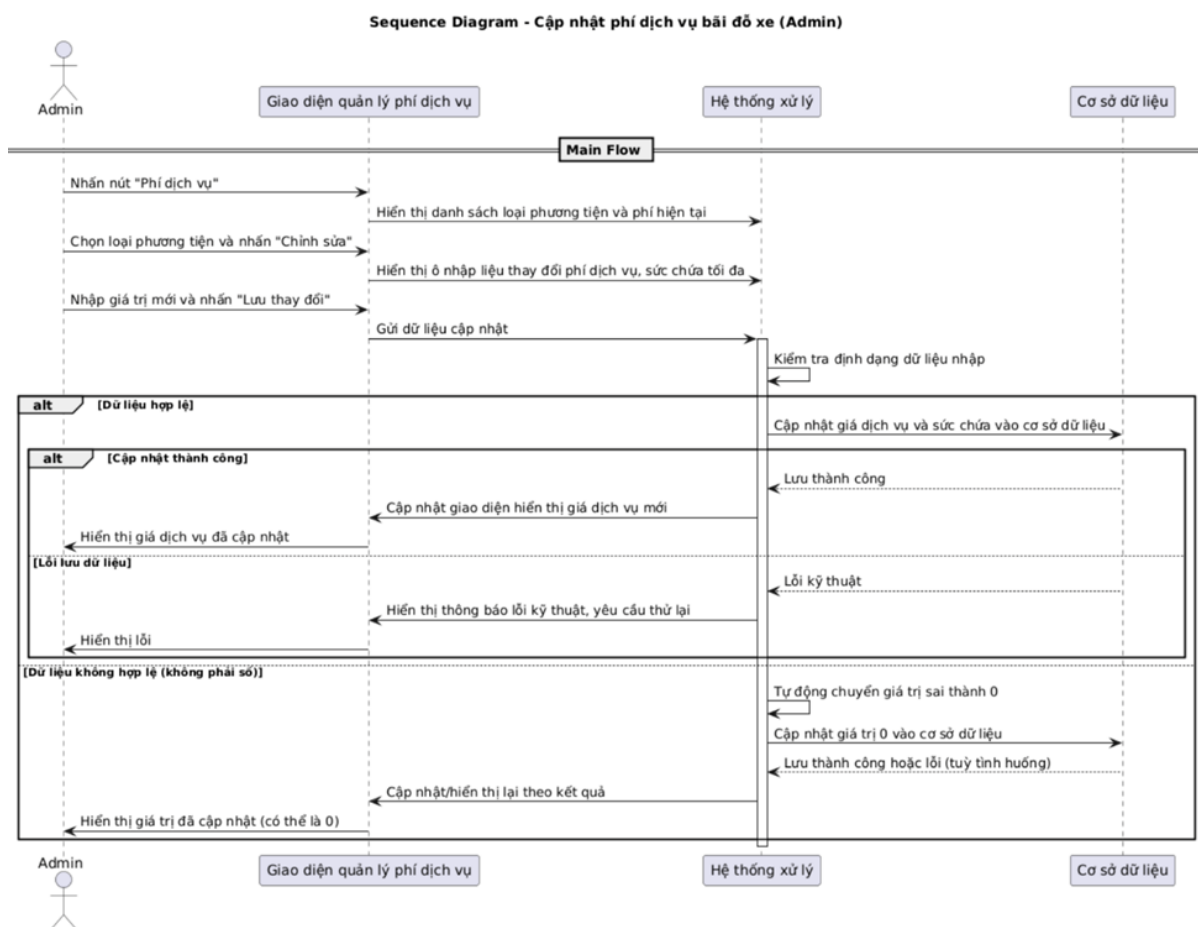
4. Admin nhập những con số muốn thay đổi và nhấn nút “Lưu thay đổi”.
5. Hệ thống xác nhận và lưu thay đổi mới về thông tin của bãi đỗ xe vào cơ sở dữ liệu.

- **Alternative Flow:**

- Bước 4: Nếu Admin nhập các ký tự không phải số, hệ thống tự động cho về 0.
- Bước 5: Nếu xảy ra lỗi khi thay đổi phí dịch vụ trong cơ sở dữ liệu, hệ thống hiển thị lỗi kỹ thuật và yêu cầu thử lại sau.

- **Post-condition:**

- Giá dịch vụ được Admin cập nhật.
- Hệ thống hiển thị giá dịch vụ tương ứng với Admin đã cập nhật.



6.1.9 Xem nhật ký hoạt động của hệ thống

- **Name:** Xem nhật ký hoạt động của hệ thống
- **Actor:** Admin
- **Goal:** Admin xem được nhật ký hoạt động của hệ thống (bao gồm tạo người dùng mới, xe nào ra xe nào vào, cập nhật phí dịch vụ,...).
- **Pre-condition:**

- Admin đã đăng nhập vào hệ thống.
- Hệ thống hiển thị màn hình chính của Admin.

• **Main Flow:**

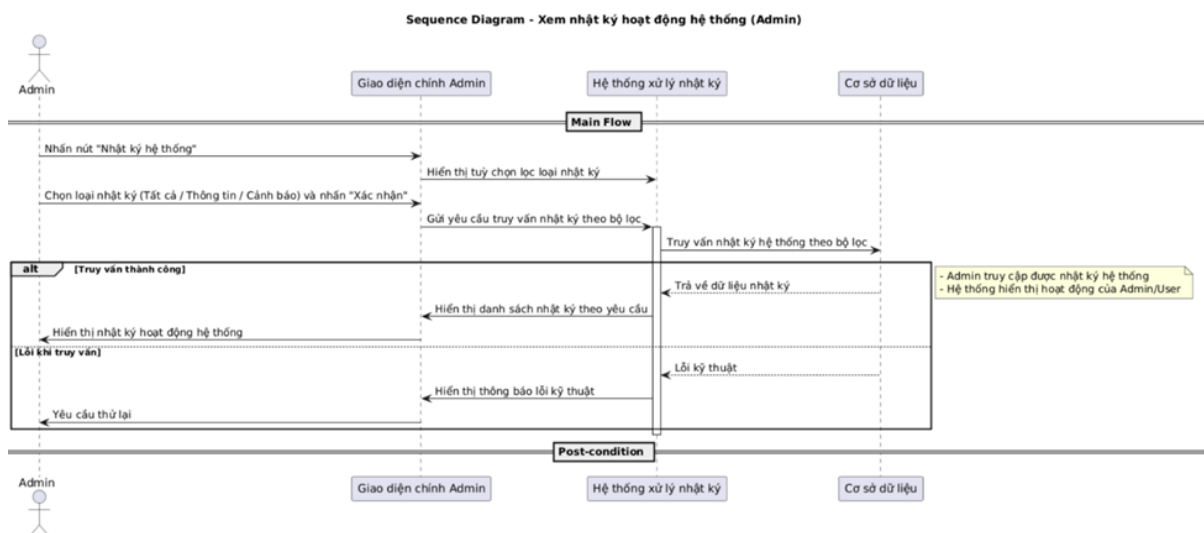
1. Admin nhấn vào nút “Nhật ký hệ thống”.
2. Admin chọn loại thông tin muốn xem (**Tất cả, Thông tin, Cảnh báo**) rồi nhấn nút “Xác nhận”.
3. Hệ thống truy vấn đến cơ sở dữ liệu và hiển thị những thông tin tương ứng với Admin đã chọn.

• **Alternative Flow:**

- Bước 3: Nếu xảy ra lỗi khi xem nhật ký hệ thống trong cơ sở dữ liệu, hệ thống hiển thị lỗi kỹ thuật và yêu cầu thử lại sau.

• **Post-condition:**

- Admin truy cập được vào nhật ký hệ thống.
- Hệ thống hiển thị những hoạt động Admin/User đã thực hiện.



6.1.10 Xuất nhật ký hoạt động của hệ thống

- **Name:** Xuất nhật ký hoạt động của hệ thống
- **Actor:** Admin
- **Goal:** Admin xuất được nhật ký hoạt động của hệ thống (được tải về dưới dạng file .csv).
- **Pre-condition:**
 - Admin đã đăng nhập vào hệ thống.
 - Hệ thống hiển thị màn hình chính của Admin.
- **Main Flow:**
 1. Admin nhấn vào nút “Nhật ký hệ thống”.

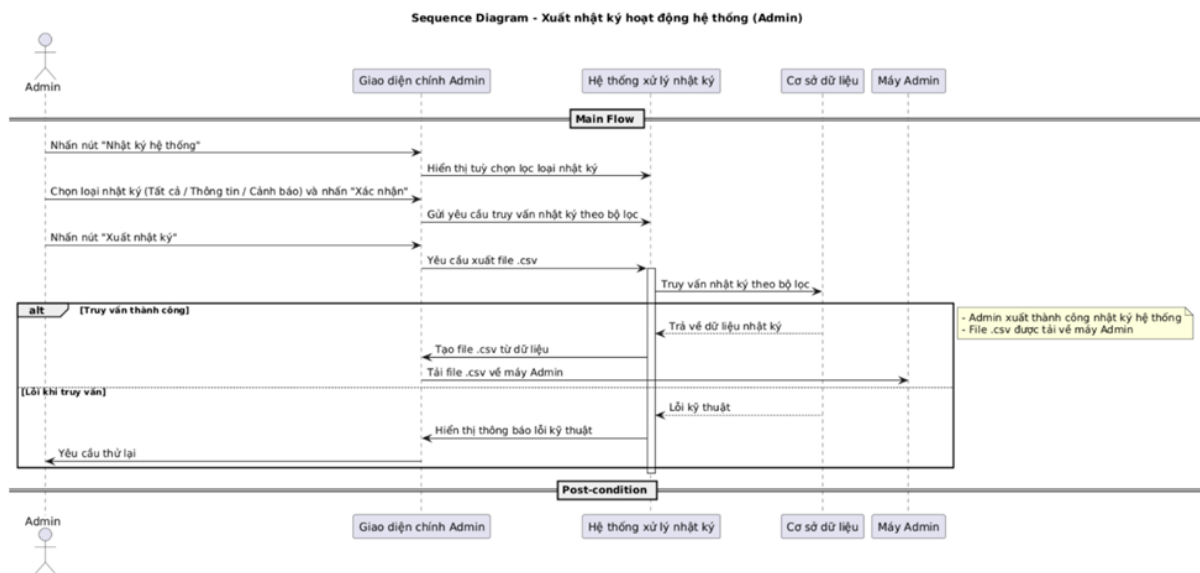
- Admin chọn loại thông tin muốn xem (**Tất cả, Thông tin, Cảnh báo**) rồi nhấn nút “Xác nhận”.
- Admin nhấn vào nút “Xuất nhật ký”.
- Hệ thống truy vấn đến cơ sở dữ liệu và tải về máy những thông tin tương ứng với Admin đã chọn dưới dạng file .csv.

- **Alternative Flow:**

- Bước 4: Nếu xảy ra lỗi khi xuất nhật ký hệ thống trong cơ sở dữ liệu, hệ thống hiển thị lỗi kỹ thuật và yêu cầu thử lại sau.

- **Post-condition:**

- Admin xuất thành công nhật ký hoạt động của hệ thống.
- Hệ thống trả về nhật ký hoạt động dưới dạng file .csv.



6.2 Tính năng của người dùng (User)

6.2.1 Đăng nhập vào tài khoản User

- **Name:** Đăng nhập tài khoản
- **Actor:** User
- **Goal:** User truy cập được vào hệ thống bằng tài khoản đã được Admin tạo từ trước.
- **Pre-condition:**
 - Tài khoản User đã được Admin đăng ký từ trước.
 - Hệ thống hiển thị giao diện đăng nhập.
- **Main Flow:**
 - User nhập thông tin đăng nhập gồm **tên đăng nhập, mật khẩu** và nhấn nút xác nhận đăng nhập.
 - Hệ thống kiểm tra tính hợp lệ của thông tin.

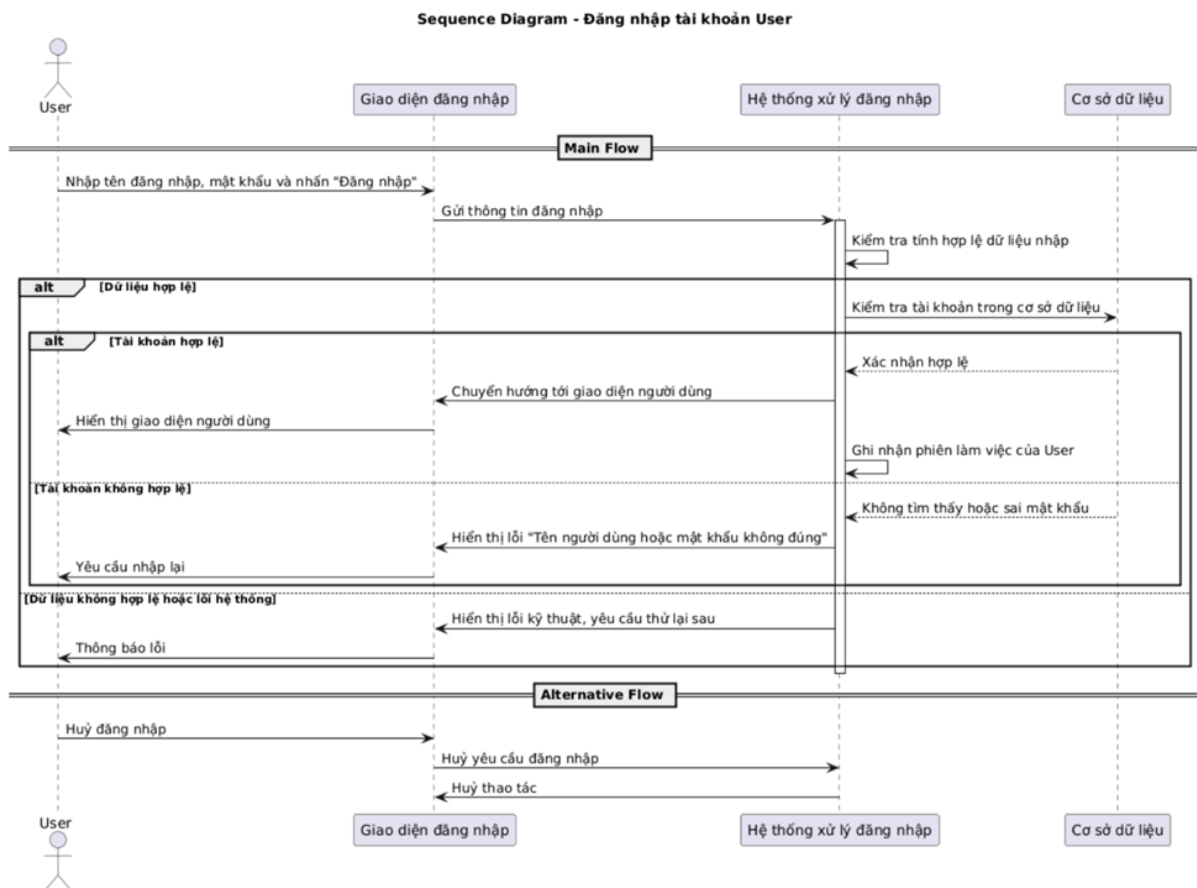
3. Hệ thống kiểm tra xem tài khoản có khớp với dữ liệu trong cơ sở dữ liệu hay không.
4. Hệ thống chuyển hướng đến giao diện người dùng của User.

- **Alternative Flow:**

- Bước 1: Nếu User hủy bỏ thao tác đăng nhập, hệ thống hủy bỏ yêu cầu đăng nhập.
- Bước 3a: Nếu thông tin đăng nhập không tồn tại trong cơ sở dữ liệu hoặc **mật khẩu** hay **tên đăng nhập** chưa đúng, hệ thống báo lỗi “Tên người dùng hoặc mật khẩu không đúng” và yêu cầu nhập lại.
- Bước 3b: Nếu có lỗi hệ thống hoặc lỗi kết nối đến cơ sở dữ liệu, hệ thống hiển thị thông báo lỗi kỹ thuật và yêu cầu thử lại sau.

- **Post-condition:**

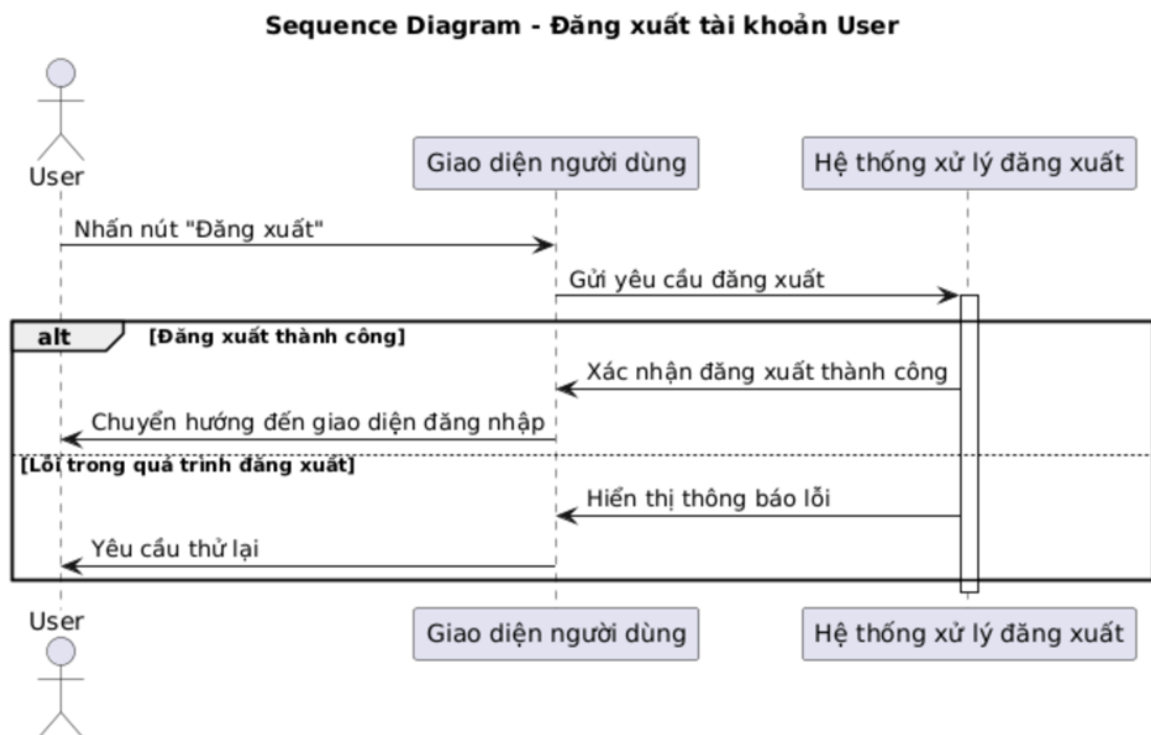
- User đăng nhập thành công và được chuyển đến giao diện người dùng.
- Hệ thống ghi nhận phiên làm việc của User.



6.2.2 Đăng xuất tài khoản User

- **Name:** Đăng xuất tài khoản User
- **Actor:** User
- **Goal:** User đăng xuất ra khỏi hệ thống.

- **Pre-condition:**
 - Tài khoản của User đã đăng nhập vào hệ thống.
 - Hệ thống hiển thị nút có chức năng đăng xuất.
- **Main Flow:**
 1. User nhấn vào nút “Đăng xuất”.
 2. Hệ thống xử lý yêu cầu và đăng xuất tài khoản User.
 3. Hệ thống chuyển hướng đến giao diện đăng nhập.
- **Alternative Flow:**
 - Bước 2: Nếu xảy ra lỗi trong quá trình đăng xuất, hệ thống hiển thị thông báo lỗi và yêu cầu thử lại.
- **Post-condition:**
 - User đã đăng xuất khỏi hệ thống.
 - Hệ thống hiển thị giao diện đăng nhập.



6.2.3 Nhận diện biển số xe bằng Camera

- **Name:** Nhận diện biển số xe bằng Camera
- **Actor:** User
- **Goal:** User nhận diện biển số xe thành công và được ghi nhận vào bãi đỗ.
- **Pre-condition:**
 - Tài khoản của User đã đăng nhập vào hệ thống.

- Hệ thống hiển thị giao diện người dùng.
- Camera hoạt động bình thường.

- **Main Flow:**

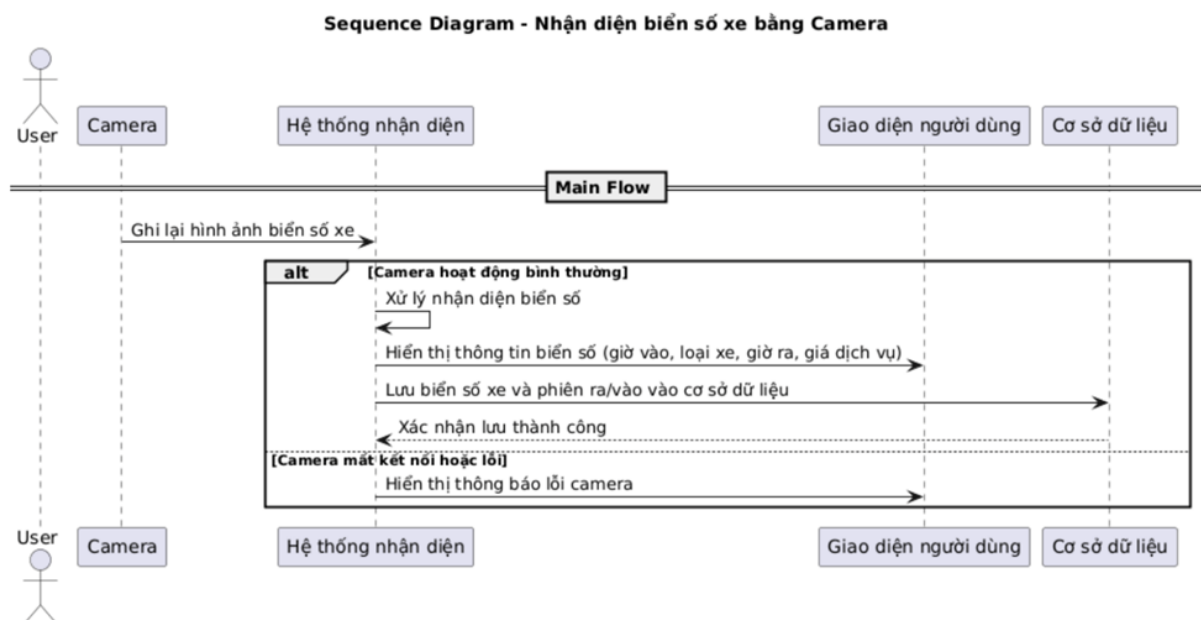
1. Camera ghi lại hình ảnh biển số xe.
2. Hệ thống ghi nhận **biển số** và hiển thị thông tin ở cạnh camera (bao gồm **giờ vào, loại xe** hoặc thêm cả **giờ ra, giá dịch vụ** nếu là phiên đi ra của xe).
3. Hệ thống lưu **biển số xe** vào cơ sở dữ liệu.

- **Alternative Flow:**

- Bước 1: Nếu camera mất kết nối hoặc bị lỗi, hệ thống hiển thị thông báo.

- **Post-condition:**

- User thành công quét biển số.
- Hệ thống ghi nhận phiên ra/vào của User.



7 Thiết kế API

Hệ thống sử dụng **FastAPI** – một framework hiện đại, có hiệu năng cao để xây dựng backend RESTful API phục vụ các chức năng quản lý bãi đỗ xe thông minh dựa trên nhận diện biển số xe. FastAPI cung cấp hiệu suất cao nhờ kiến trúc bất đồng bộ (asynchronous), tự động kiểm tra và xác thực dữ liệu, đồng thời hỗ trợ tự động sinh tài liệu API với OpenAPI/Swagger. Điều này giúp FastAPI được lựa chọn dựa trên các tiêu chí:

- Đáp ứng tốt các yêu cầu real-time.
- Dễ tích hợp với mô hình AI.
- Quản lý dữ liệu và API rõ ràng, dễ dàng mở rộng.
- Tài liệu được sinh tự động tiện cho việc bảo trì và phát triển.

- Swagger UI tại địa chỉ <http://127.0.0.1:8000/docs> được tích hợp sẵn, giúp dễ dàng theo dõi luồng dữ liệu và kiểm thử các endpoint.

Các nhóm API chính trong hệ thống:

1. Admin:

- POST /admin/create_user: Tạo người dùng mới.
- PUT /admin/modify_user_info/{username}: Chỉnh sửa thông tin người dùng.
- GET /admin/get_all_users: Lấy thông tin tất cả người dùng.
- DELETE /admin/delete_user/{username}: Xóa người dùng.
- GET /admin/get_parking_config: Lấy cấu hình bãi đỗ.
- PUT /admin/update_parking_config/{id}: Chỉnh sửa thông tin bãi đỗ.
- DELETE /admin/update_parking_config/{id}: Xóa thông tin bãi đỗ.
- POST /admin/create_parking_config: Tạo mới thông tin bãi đỗ.

2. User:

- POST /login: Đăng nhập.
- GET /get_user_info/{username}: Lấy thông tin người dùng.

3. Parking:

- POST /auto_check: Kiểm tra người dùng check-in hay check-out.
- GET /get_parking_session/{license_plate}: Lấy thông tin phiên đỗ xe.
- GET /get_all_parking_sessions: Lấy thông tin tất cả phiên đỗ xe.
- PUT /update_parking_session/{license_plate}: Cập nhật phiên đỗ xe.
- DELETE /delete_parking_session/{license_plate}: Xóa phiên đỗ xe.

Các cấu trúc API được tạo ra, tổ chức một cách rõ ràng theo từng vai trò và chức năng, đảm bảo việc mở rộng, bảo trì hệ thống trong tương lai trở nên dễ dàng.

8 Triển khai và vận hành

8.1 Môi trường triển khai

8.1.1 Môi trường phát triển (Development)

- **Hạ tầng:** Máy cá nhân hoặc máy chủ cục bộ.
- **Triển khai dạng container:**
 - **Services:** Chạy FastAPI kèm OpenCV + PyTorch + YOLOv5 để xử lý và nhận diện biển số xe.
 - **Backend:** FastAPI + SQLAlchemy + SQLite để quản lý người dùng, bãi đỗ xe, và các phiên gửi xe.
 - **Frontend:** Next.js cung cấp giao diện người dùng.
- **Công cụ hỗ trợ:**

- Sử dụng Docker Compose để chạy đồng thời các container.
- Quản lý phụ thuộc môi trường dễ dàng, nhanh chóng thử nghiệm tính năng mới.

8.1.2 Môi trường kiểm thử (Staging/Test)

- **Triển khai:** Trên máy chủ riêng hoặc cloud (như AWS, GCP, hoặc server ảo).
- **Cấu hình:** Tương tự môi trường production để đảm bảo các lỗi chỉ xảy ra ở môi trường này, không ảnh hưởng môi trường thật.
- **Dữ liệu:** Sử dụng dữ liệu mô phỏng để kiểm thử logic hệ thống, tránh rò rỉ dữ liệu thật.
- **Kiểm thử tích hợp:** Đảm bảo frontend, backend và dịch vụ nhận diện hoạt động mượt mà với nhau.

8.1.3 Môi trường sản xuất (Production)

- **Triển khai:** Trên cloud hoặc server nội bộ.
- **Bảo mật:** Triển khai HTTPS, giới hạn truy cập theo IP nếu cần.
- **Reverse Proxy:** Sử dụng Nginx để phân phối lưu lượng đến các container:
 - 8001: Dịch vụ nhận diện biển số.
 - 8000: Backend quản lý hệ thống.
 - 80: Frontend giao diện người dùng.

8.2 Công cụ đi kèm

- **Frontend:** [Next.js](#).
- **Backend:** [FastAPI](#).
- **Cơ sở dữ liệu:** [SQLite](#).
- **ORM:** [SQLAlchemy](#).
- **Services:** [YOLOv5](#), FastAPI, WebSocket.
- **Triển khai:** Docker, Docker Compose.

8.3 Chiến lược backup, giám sát và bảo trì

8.3.1 Backup dữ liệu

- Thiết lập cơ chế sao lưu định kỳ cho cơ sở dữ liệu SQLite, lưu trữ tại các dịch vụ cloud an toàn hoặc máy chủ nội bộ.
- Đảm bảo khả năng phục hồi dữ liệu nhanh chóng trong trường hợp có sự cố.

8.3.2 Giám sát hệ thống

- Theo dõi log của các container để phát hiện và xử lý sự cố kịp thời.

- Giám sát hiệu suất của các API, đặc biệt là thời gian phản hồi và tải hệ thống.
- Kiểm tra hiệu suất của dịch vụ nhận diện biển số khi xử lý nhiều luồng video đồng thời.

8.3.3 Bảo trì hệ thống

- Cập nhật định kỳ các thư viện và framework (FastAPI, Next.js, YOLOv5,...) để đảm bảo bảo mật và hiệu suất.
- Kiểm tra và áp dụng các bản vá bảo mật cho hệ điều hành và các dịch vụ liên quan.
- Dọn dẹp dữ liệu không cần thiết (như ảnh cũ, phiên gửi xe đã kết thúc) để tối ưu hóa dung lượng lưu trữ và hiệu suất hệ thống.

9 Hiệu suất và khả năng mở rộng hệ thống

9.1 Hiệu suất hệ thống

Hệ thống Park-Scan được thiết kế để đảm bảo hiệu suất cao trong việc quản lý bãi đỗ xe thông minh, với các đặc điểm nổi bật sau:

- **Nhận diện biển số xe thời gian thực:** Sử dụng mô hình YOLOv5 kết hợp với OpenCV và PyTorch, hệ thống có khả năng nhận diện biển số xe từ luồng video webcam với độ trễ thấp, đảm bảo quá trình check-in/check-out diễn ra nhanh chóng và chính xác.
- **Xử lý bất đồng bộ:** FastAPI hỗ trợ xử lý bất đồng bộ (async/await), cho phép hệ thống xử lý nhiều yêu cầu đồng thời mà không ảnh hưởng đến hiệu suất tổng thể.
- **Tối ưu hóa tài nguyên:** Hệ thống sử dụng cơ chế phát hiện chuyển động để kích hoạt quá trình nhận diện biển số, giúp tiết kiệm tài nguyên xử lý khi không có hoạt động trong khung hình.
- **Phản hồi nhanh chóng:** Thời gian phản hồi của API được tối ưu hóa, đảm bảo người dùng và quản trị viên nhận được thông tin kịp thời về trạng thái bãi đỗ và các phiên gửi xe.

9.2 Khả năng mở rộng

Hệ thống Park-Scan được xây dựng với kiến trúc linh hoạt, cho phép mở rộng dễ dàng theo nhu cầu sử dụng:

- **Kiến trúc microservices:** Các thành phần Services, Backend và Frontend được triển khai dưới dạng các container Docker riêng biệt, cho phép mở rộng từng thành phần một cách độc lập.
- **Triển khai linh hoạt:** Hệ thống có thể được triển khai trên nhiều môi trường khác nhau, từ máy chủ cục bộ đến các dịch vụ cloud như AWS, Azure hoặc Google Cloud, tùy thuộc vào quy mô và yêu cầu cụ thể.
- **Cơ sở dữ liệu linh hoạt:** Mặc dù hiện tại sử dụng SQLite cho môi trường phát

triển và thử nghiệm, hệ thống có thể dễ dàng chuyển sang các hệ quản trị cơ sở dữ liệu mạnh hơn như PostgreSQL hoặc MySQL để đáp ứng nhu cầu lưu trữ và truy vấn dữ liệu lớn hơn.

- **Tích hợp với các hệ thống khác:** Hệ thống có khả năng tích hợp với các dịch vụ và thiết bị khác như cảm biến IoT, cổng barrier tự động, hệ thống thanh toán điện tử, giúp mở rộng chức năng và nâng cao trải nghiệm người dùng.
- **Hỗ trợ đa bãi đỗ:** Hệ thống được thiết kế để quản lý nhiều bãi đỗ xe cùng lúc, với khả năng cấu hình riêng biệt cho từng bãi, đáp ứng nhu cầu của các tổ chức có nhiều địa điểm đỗ xe.

Hệ thống Park-Scan, với kiến trúc linh hoạt và hiệu suất cao, sẵn sàng đáp ứng nhu cầu quản lý bãi đỗ xe thông minh trong các môi trường đô thị hiện đại, đồng thời dễ dàng mở rộng và tích hợp với các công nghệ mới trong tương lai.